IMPLEMENT CONFUSION MATRIX

Step-1 : import necessary libraries

```
import numpy as np
```

//numpy for array creation

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```

//confusion_matrix for calculation

//ConfusionMatrixDisplay for visualization

```
import matplotlib.pyplot as plt
```

//for plotting

Step -2 : Define sample data

```
# Actual labels (ground truth)
y_true = np.array([1, 0, 1, 1, 0, 1, 0, 1, 0, 0])

# Predicted labels from a classifier
y_pred = np.array([1, 0, 0, 1, 0, 1, 0, 1, 1, 0])
```

Step 3: Compute the confusion matrix

```
#Use the confusion_matrix() function, passing true and predicted labels.
cm = confusion_matrix(y_true, y_pred)
print(cm)
```

output: [[4 1]

[1 4]]

Top Left - True Negative (TN): Tells us how many times a model correctly classifies a negative sample as Negative - 4 negative cases

Top Right - False Positive (FP): Tells us how many times a model incorrectly classifies a negative sample as Positive - 1 positive case

Bottom Left - False Negative (FN): Tells us how many times a model incorrectly classifies a positive sample as Negative - 1 negative case

Bottom Right - True Positive (TP): Tells us how many times a model correctly classifies a positive sample as Positive - 4 positive cases

Step 4: Visualize the confusion matrix

For better readability, especially with multi-class problems, visualize the matrix using ConfusionMatrixDisplay

```
# Define display labels for clarity

class_labels = ['Class 0', 'Class 1']


# Create the display object

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_labels)


# Plot the matrix

disp.plot(cmap=plt.cm.Blues)

plt.title('Confusion Matrix')

plt.show()
```

Implementing performance metrics for classification model

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# True labels and predicted labels
y_true = [0, 1, 0, 1, 1, 0, 1, 0]   # actual values
y_pred = [0, 1, 0, 0, 1, 0, 1, 1]   # predicted values

# Performance metrics
print("Accuracy :", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall   :", recall_score(y_true, y_pred))
print("F1 Score :", f1_score(y_true, y_pred))

# Confusion matrix
print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
```

output:

```
Accuracy : 0.75
Precision: 0.75
Recall   : 0.75
F1 Score : 0.75
Confusion Matrix:
 [[3 1]
 [1 3]]
```

PROGRAM 8 - Implementing ensemble learning

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.metrics import accuracy_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Base models
model1 = LogisticRegression(max_iter=1000)
model2 = DecisionTreeClassifier()
model3 = KNeighborsClassifier()

# Ensemble (Voting Classifier)
ensemble = VotingClassifier(estimators=[
    ('lr', model1), ('dt', model2), ('knn', model3)
], voting='hard')

# Train ensemble
ensemble.fit(X_train, y_train)

# Predictions
y_pred = ensemble.predict(X_test)
```

```
# Accuracy
print("Ensemble Accuracy:", accuracy_score(y_test, y_pred))
```

output:

```
Ensemble Accuracy: 1.0
```

Program 9:

Implement AdaBosst algorithm

The AdaBoost (Adaptive Boosting) algorithm can be implemented in Python using the sklearn library, which provides a convenient AdaBoostClassifier class.

#1. Import necessary libraries:

```
from sklearn.ensemble import AdaBoostClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

from sklearn.datasets import make_classification # For creating a sample dataset
```

#2. Create a sample dataset:

```
X, y = make_classification(n_samples=100, n_features=10, n_classes=2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3. Define the weak learner and the AdaBoost model:

A common choice for the weak learner in AdaBoost is a Decision Stump, which is a Decision Tree with a maximum depth of 1.

```
# Define the weak learner (Decision Stump)

weak_learner = DecisionTreeClassifier(max_depth=1)
```

```
# Define the AdaBoostClassifier model

# n_estimators: number of weak learners to train
```

learning_rate: shrinks the contribution of each weak learner

adaboost_model = AdaBoostClassifier(estimator=weak_learner, n_estimators=50, learning_rate=1.0, random_state=42)

4. Train the AdaBoost model:

adaboost_model.fit(X_train, y_train)

5. Make predictions and evaluate the model:

y_pred = adaboost_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f"AdaBoost Model Accuracy: {accuracy:.4f}")

This implementation utilizes sklearn's pre-built classes, simplifying the process. The AdaBoostClassifier handles the iterative training of weak learners, weight updates, and combines their predictions for the final output.

PROGRAM 10: IMPLEMENT BAGGING USING DECISION TREES

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

# Base model
base_model = DecisionTreeClassifier()

# Bagging with 10 decision trees
bag_model = BaggingClassifier(estimator=base_model, n_estimators=10, random_state=0)

# Train model
bag_model.fit(X_train, y_train)

# Predict
y_pred = bag_model.predict(X_test)

# Accuracy
print("Bagging Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

```
Bagging Accuracy: 0.9555555555555556
```

Steps:

1. Loads the Iris dataset.
2. Uses Decision Tree as the base model.
3. Creates a Bagging ensemble with 10 trees.
4. Trains and prints accuracy.